

Deep Learning

4. Fonctions de perte, mise à l'échelle, régularisation

Romain Tavenard

Optimisation en général

- **Objectif** : ajuster les paramètres du modèle afin de minimiser l'erreur
- Mesurer l'erreur
 - Via une fonction de coût / fonction de perte
 - Exemple typique : l'erreur quadratique moyenne en régression

Fonctions de perte

- Pour entraîner un modèle, il faut **mesurer à quel point il se trompe** : c'est le rôle de la fonction de perte $\mathcal{L}$
- Pertes usuelles
 - **Erreur quadratique moyenne (MSE)** pour la régression
(pénalise les grandes erreurs)

$$\mathcal{L}(x_i, y_i; \theta) = (m(x_i; \theta) - y_i)^2$$

- **Entropie croisée** pour la classification
(pénalise les prédictions erronées mais confiantes)

$$\mathcal{L}(x_i, y_i; \theta) = -\log P_{\theta}(y = y_i \mid x_i)$$

- Exigence technique pour la descente de gradient :
 $\mathcal{L}$ doit être dérivable

Mise à l'échelle

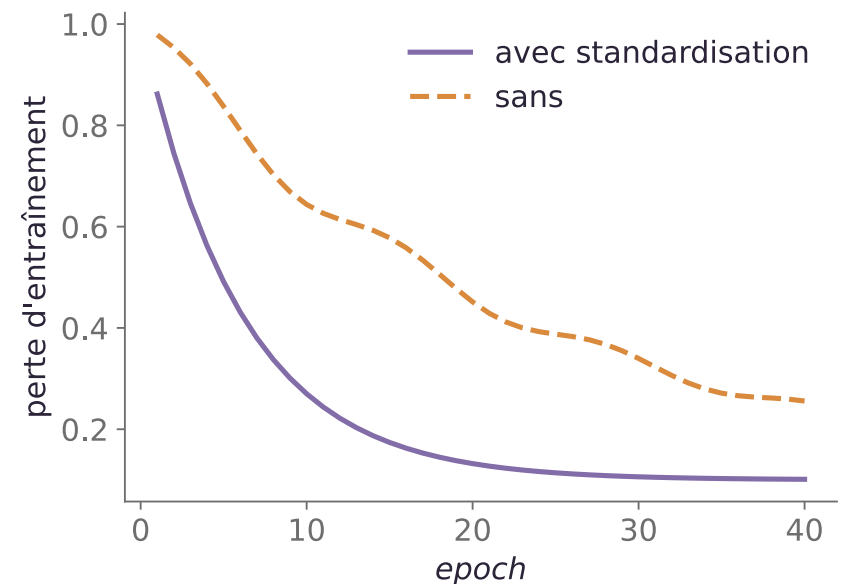
- En pratique, **mettre à l'échelle les caractéristiques d'entrée**

est crucial pour un entraînement stable

- Centrer-réduire chaque caractéristique

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j}$$

- ou mise à l'échelle spécifique au domaine (par ex. images dans $[0, 1]$)



- **Intuition** : garder l'“intensité” du signal (la variance) **constante** à travers les couches
 - trop grande → les activations explosent
 - trop petite → elles s'annulent (disparition des gradients, *cf.* cours suivant)
- **D'où viennent ces formules** : un neurone somme n_{in} entrées → la variance est multipliée par n_{in} → pour la garder stable :

$$\text{Var}(w) = \frac{1}{n_{\text{in}}}$$

Deux méthodes classiques = raffinements de $\text{Var}(w) = 1/n_{\text{in}}$:

- **Xavier / Glorot** (tanh, sigmoïde) :

$$w \sim \mathcal{U} \left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}} \right)$$

- stable en propagation avant (n_{in}) **et** arrière (n_{out})
- suppose φ linéaire près de 0

- **He** (ReLU) :

$$w \sim \mathcal{N} \left(0, \sqrt{\frac{2}{n_{\text{in}}}} \right)$$

- ReLU supprime **la moitié** du signal $\rightarrow \times 2$ sur la variance

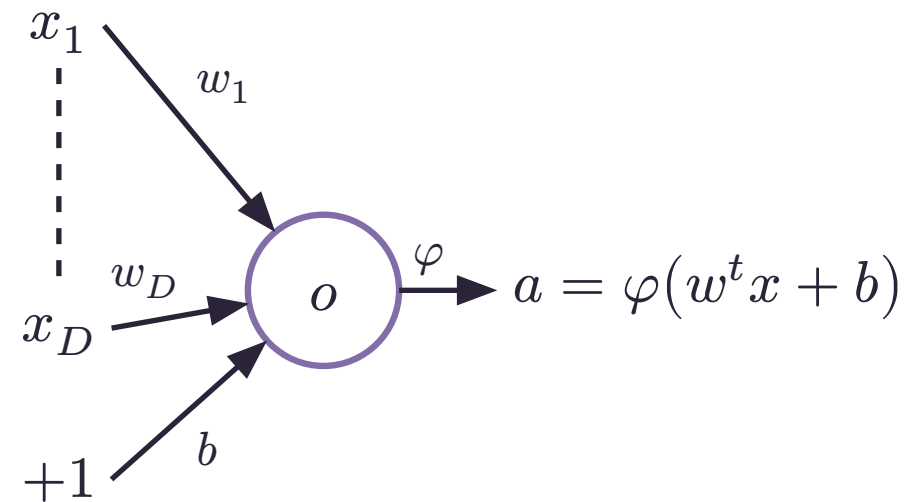
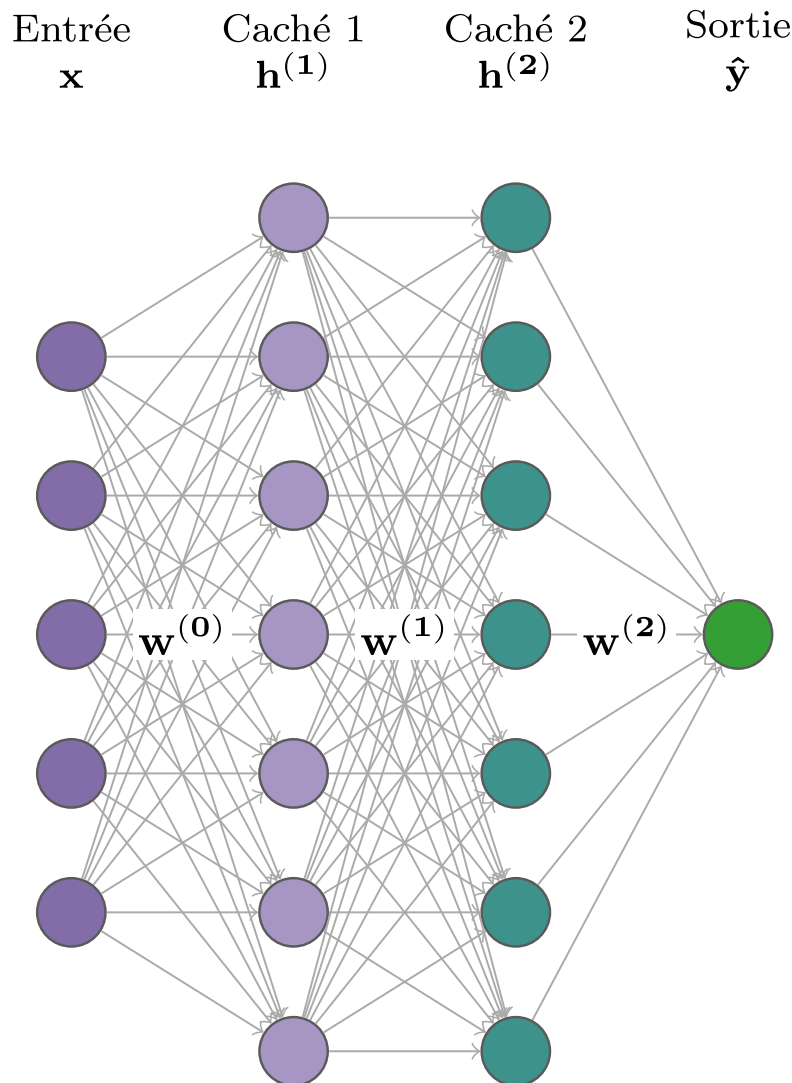
- **L'initialisation ne fixe l'échelle qu'à $t = 0$**
- **Batch Normalization** : même objectif que l'initialisation, mais **imposé pendant l'entraînement** : à **chaque** *mini-batch*

$$\hat{z}^{(l)} = \frac{z^{(l)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}$$

- ▶ puis une échelle γ et un décalage β apprenables
- **Mode entraînement vs évaluation** :
 - ▶ **Entraînement** : $\mu_{\mathcal{B}}, \sigma_{\mathcal{B}}^2$ = statistiques du *mini-batch* courant
 - ▶ **Évaluation** : les statistiques de *mini-batch* ne sont pas fiables (taille de batch 1, pas de batch)
→ moyenne glissante μ, σ^2 issue de l'entraînement, figée au moment du test

Disparition des gradients

Réseaux de neurones et rétropropagation



$$\frac{\partial a^{(l)}}{\partial o^{(l)}} = \varphi'(o^{(l)})$$

$$\frac{\partial o^{(l)}}{\partial a^{(l-1)}} = w^{(l-1)}$$

Réseaux de neurones et rétropropagation

Disparition des gradients

$$\frac{\partial \mathcal{L}}{\partial w^{(2)}} = \frac{\partial \mathcal{L}}{\partial a^{(3)}} \frac{\partial a^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial w^{(2)}}$$

$$\frac{\partial \mathcal{L}}{\partial w^{(1)}} = \frac{\partial \mathcal{L}}{\partial a^{(3)}} \frac{\partial a^{(3)}}{\partial o^{(3)}} \frac{\partial o^{(3)}}{\partial a^{(2)}} \frac{\partial a^{(2)}}{\partial o^{(2)}} \frac{\partial o^{(2)}}{\partial w^{(1)}}$$

$$\frac{\partial \mathcal{L}}{\partial w^{(0)}} = \frac{\partial \mathcal{L}}{\partial a^{(3)}} \underbrace{\frac{\partial a^{(3)}}{\partial o^{(3)}}}_{\varphi'(o^{(3)})} \frac{\partial o^{(3)}}{\partial a^{(2)}} \underbrace{\frac{\partial a^{(2)}}{\partial o^{(2)}}}_{\varphi'(o^{(2)})} \frac{\partial o^{(2)}}{\partial a^{(1)}} \underbrace{\frac{\partial a^{(1)}}{\partial o^{(1)}}}_{\varphi'(o^{(1)})} \frac{\partial o^{(1)}}{\partial w^{(0)}}$$

Réseaux plus profonds et disparition des gradients

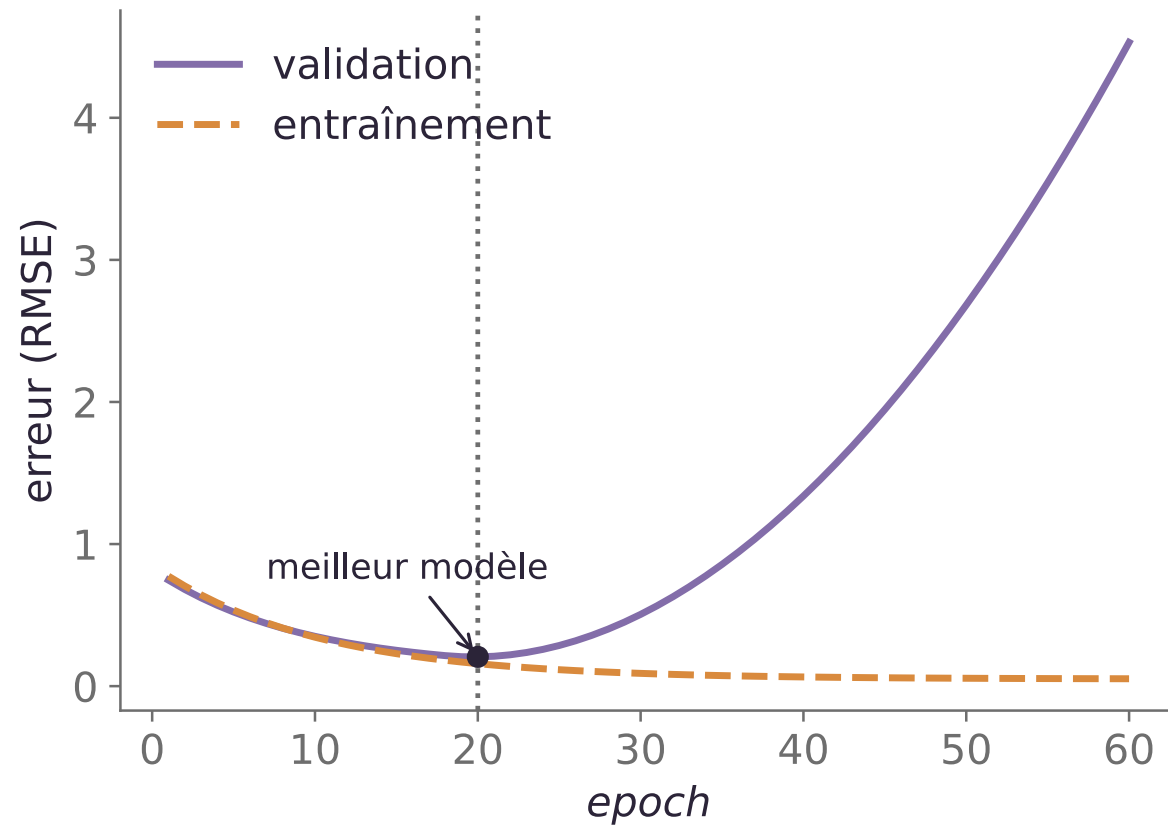
- Des réseaux plus profonds = une compréhension de plus haut niveau
- Mais un problème apparaît : la **disparition des gradients**
- **Intuition :**
 - le gradient remonte à travers **chaque** couche suivante
 - un facteur multiplicatif par couche
 - beaucoup de facteurs < 1
 - le produit tend vers 0
 - les premières couches apprennent à peine
 - **ReLU aide** : $\varphi' = 1$ (côté actif) → facteur 1, pas d'annulation

Régularisation

- Optimisation (SGD) pour minimiser une fonction de perte
 - Des réseaux plus grands et plus profonds améliorent les performances (à l'entraînement)
 - Risque de surapprentissage

$$\arg \min_{\theta} \sum_{(x_i, y_i) \in \mathcal{D}_t} \mathcal{L}(x_i, y_i; \theta) \neq \arg \min_{\theta} \mathbb{E}_{x, y \sim \mathcal{D}} \mathcal{L}(x, y; \theta)$$

- Astuces de régularisation
 - Early Stopping (cf. gradient boosting)
 - Dropout (à rapprocher des forêts aléatoires)



S'arrêter là où l'erreur de validation est minimale

- À chaque *mini-batch*, on **désactive** aléatoirement une fraction des neurones
- Tous les neurones finissent par être entraînés au cours du processus complet
- Dans le même esprit que les forêts aléatoires (sous-échantillonnage des caractéristiques)

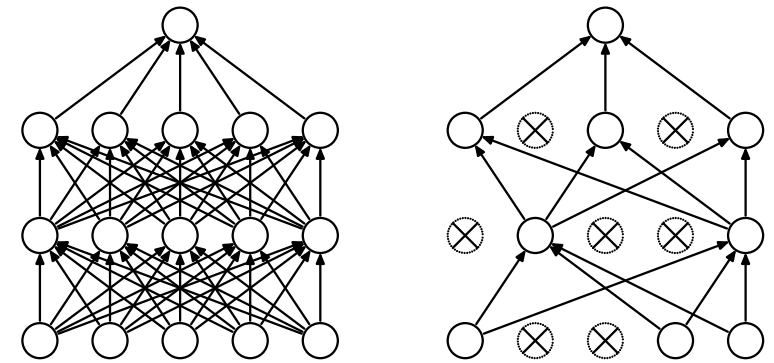


Illustration du Dropout — à gauche : réseau complet ; à droite : même réseau avec 40% des neurones désactivés pour un mini-batch.

Source : [Srivastava et al., 2014]

- Fonctions de perte
 - Erreur quadratique moyenne pour la régression
 - Perte logistique pour la classification
- Garder des activations bien mises à l'échelle
 - standardisation de l'entrée
 - initialisation → échelle au démarrage
 - BatchNorm → échelle pendant l'entraînement
- Disparition des gradients
 - réseaux plus profonds → le produit tend vers 0
 - ReLU aide (facteur = 1 côté actif)
- Régularisation
 - Early Stopping
 - Dropout